

ONEDRAFT

CAD-AI — Aria Powered

Executable Application & Domain Service Layer

Version 1.0

Status: Implementation Specification

Predecessor: Database Schema & OpenAPI Contract v0.1

Architecture: Executable Application Stack & Implementation Architecture v0.2

Database: PostgreSQL

API: REST / JSON

API Version: /api/v1

Queue: Redis-backed asynchronous jobs

Workers: Python worker processes

Domain Model: Versioned OneDraft Project Model

AI Layer: Aria Powered

Persistence Boundary: Repository + Unit of Work

Execution Boundary: Application Commands

Event Boundary: Transactional Outbox

Primary Objective: Convert the established OneDraft persistence/API contract into an executable application foundation.

1. IMPLEMENTATION STATE

The OneDraft system has now progressed beyond conceptual architecture.

The following architectural layers are established:

Product definition

System architecture

Technology architecture

Domain model

Database schema

OpenAPI contract

Versioning model

AI command boundary

Regulatory model

Drawing model

Redline model

Validation model

Document provenance

Acceptance workflow

The next layer is the executable service layer.

The purpose of this stage is to establish the application runtime capable of receiving API requests, executing domain operations, persisting transactional state, creating revisions, publishing events, and dispatching asynchronous work.

2. APPLICATION STACK

The executable OneDraft stack shall be:

```
CLIENT
  ↓
API GATEWAY / HTTP
  ↓
FASTAPI APPLICATION
  ↓
APPLICATION SERVICES
  ↓
DOMAIN COMMANDS
  ↓
DOMAIN MODEL
  ↓
UNIT OF WORK
  ↓
REPOSITORIES
  ↓
POSTGRESQL
```

Asynchronous execution branches from the application layer:

```
APPLICATION COMMAND
  ↓
TRANSACTIONAL OUTBOX
  ↓
REDIS / JOB QUEUE
  ↓
WORKER
  ↓
DOMAIN SERVICE
  ↓
POSTGRESQL / OBJECT STORAGE
```

Aria operates above the command boundary:

```
USER REQUEST
  ↓
ARIA ORCHESTRATOR
  ↓
INTENT INTERPRETATION
  ↓
COMMAND
  ↓
APPLICATION SERVICE
  ↓
DOMAIN MODEL
```

Aria never bypasses the application boundary.

3. RUNTIME REPOSITORY

The executable implementation shall extend the existing OneDraft repository.

The initial runtime structure shall be:

```
apps/
├── api/
│   ├── main.py
│   ├── dependencies.py
│   ├── routes/
│   │   ├── projects.py
│   │   ├── requirements.py
│   │   ├── buildings.py
│   │   ├── levels.py
│   │   ├── spaces.py
│   │   ├── elements.py
│   │   ├── commands.py
│   │   ├── revisions.py
│   │   ├── redlines.py
│   │   ├── validation.py
│   │   ├── drawings.py
│   │   ├── documents.py
│   │   ├── acceptance.py
│   │   └── jobs.py
│   └── schemas/
│       └── ...
├── worker/
│   ├── worker.py
│   ├── jobs/
│   │   ├── geometry.py
│   │   ├── validation.py
│   │   ├── drawings.py
│   │   ├── documents.py
│   │   └── aria.py
│   └── ...
└── services/
```

- project_model/
- aria/
- geometry/
- compliance/
- validation/
- documentation/
- acceptance/

```
packages/  
├── domain/  
│   ├── commands.py  
│   ├── entities.py  
│   ├── value_objects.py  
│   ├── events.py  
│   ├── errors.py  
│   └── services.py  
├── persistence/  
│   ├── database.py  
│   ├── repositories.py  
│   ├── unit_of_work.py  
│   └── models/  
└── infrastructure/  
    ├── queue.py  
    ├── storage.py  
    ├── logging.py  
    └── configuration.py
```

The exact package organization may evolve, but domain logic must remain separated from transport and infrastructure concerns.

4. CORE IMPLEMENTATION ARTIFACTS

The immediate executable foundation shall contain:

```
commands.py  
database.py  
repositories.py  
unit_of_work.py  
main.py  
projects.py  
worker.py
```

These files form the first application runtime.

They are not independent utilities.

They form a single execution chain:

```
HTTP REQUEST  
↓  
main.py  
↓
```

projects.py
↓
commands.py
↓
unit_of_work.py
↓
repositories.py
↓
database.py
↓
POSTGRESQL

Asynchronous work proceeds through:

APPLICATION
↓
EVENT / JOB
↓
worker.py
↓
DOMAIN SERVICE

5. COMMAND MODEL

commands.py becomes the machine-readable representation of operations permitted against the Project Model.

Initial command classes:

CreateProjectCommand
CreateRequirementCommand
CreateBuildingCommand
CreateLevelCommand
CreateSpaceCommand
CreateElementCommand
ModifyElementCommand
MoveElementCommand
DeleteElementCommand
CreateRedlineCommand
ResolveRedlineCommand
GenerateDrawingsCommand
RunValidationCommand
GenerateDocumentPackageCommand
AcceptRevisionCommand

Every command must identify:

command_id
project_id
actor_id
expected_revision
operation
parameters

Where appropriate:

```
preserve_constraints
affected_objects
source
reason
```

6. COMMAND IMMUTABILITY

Commands are immutable instructions.

Once accepted by the application layer, the original command payload shall not be modified.

Execution results are recorded separately.

Conceptually:

```
COMMAND
  ↓
EXECUTION
  ↓
RESULT
```

The command represents what was requested.

The result represents what actually occurred.

7. EXPECTED REVISION

Every mutating command should support optimistic concurrency.

Example:

```
expected_revision = 14
```

The application verifies:

```
current_revision == expected_revision
```

If the current revision differs:

```
REVISION_CONFLICT
```

is returned.

This prevents an outdated client or AI process from silently modifying newer project state.

8. COMMAND VALIDATION

Commands are validated before execution.

Validation occurs in layers:

1. Request schema validation
2. Authentication
3. Authorization
4. Project existence
5. Revision currency
6. Object existence
7. Parameter validation
8. Constraint validation
9. Domain validation
10. Execution

No domain mutation occurs before these gates succeed.

9. DATABASE CONNECTION LAYER

`database.py` owns PostgreSQL connectivity.

The database layer shall provide:

engine
session factory
connection lifecycle
transaction support
health check

The database connection string is supplied through configuration.

It shall never be hard-coded.

Example configuration:

`DATABASE_URL`

The application must fail clearly during startup if a required database configuration is absent.

10. DATABASE SESSION MANAGEMENT

Each HTTP request requiring persistence receives a controlled database session.

Conceptually:

```
REQUEST
↓
SESSION
↓
UNIT OF WORK
↓
TRANSACTION
↓
COMMIT / ROLLBACK
↓
SESSION CLOSE
```

Sessions must not be retained globally across requests.

Workers receive independent sessions.

11. UNIT OF WORK

`unit_of_work.py` becomes the transactional boundary for domain mutations.

Conceptually:

```
with unit_of_work() as uow:
    project = uow.projects.get(project_id)

    command.execute(project)

    uow.commit()
```

The Unit of Work coordinates:

```
repositories
transactions
events
outbox records
```

12. UNIT OF WORK COMMIT RULE

A successful command follows:

```
BEGIN
↓
LOAD
↓
```



```
VALIDATE
↓
MUTATE
↓
RECORD CHANGE
↓
CREATE REVISION
↓
CREATE EVENT
↓
CREATE OUTBOX MESSAGE
↓
COMMIT
```

Failure results in:

ROLLBACK

No partial domain mutation may remain.

13. REPOSITORY LAYER

`repositories.py` provides persistence access without exposing database implementation details to domain services.

Initial repositories:

```
OrganizationRepository
UserRepository
ProjectRepository
RequirementRepository
BuildingRepository
LevelRepository
SpaceRepository
ElementRepository
AssemblyRepository
ConstraintRepository
RelationshipRepository
ModelVersionRepository
RevisionRepository
ChangeRecordRepository
CommandRepository
RedlineRepository
CodeManifestRepository
ValidationRepository
DrawingRepository
DocumentPackageRepository
AcceptanceRepository
EventRepository
OutboxRepository
```

14. REPOSITORY RESPONSIBILITY

Repositories are responsible for:

- retrieve
- insert
- update
- query
- existence checks

Repositories are not responsible for:

- HTTP
- AI interpretation
- authorization policy
- drawing generation
- geometry computation
- business workflow orchestration

Those concerns remain above the persistence layer.

15. PROJECT REPOSITORY

The ProjectRepository shall provide operations equivalent to:

- create()
- get()
- get_required()
- list_for_organization()
- update()
- exists()

The repository must enforce organization/project boundaries supplied by the application service.

A project must never be retrieved without sufficient tenancy context.

16. ELEMENT REPOSITORY

The ElementRepository is a high-frequency repository.

It shall support:

- get()
- get_many()
- get_children()
- get_by_level()
- get_by_type()
- get_by_parent()
- get_by_host()
- create()

```
update()  
delete()
```

Queries must be indexed according to the database strategy established in v0.1.

17. REVISION REPOSITORY

The RevisionRepository provides:

```
get_current()  
get()  
create()  
list()
```

The repository must preserve monotonically increasing project revision numbers.

A revision cannot be silently overwritten.

18. MODEL VERSION REPOSITORY

Model versions represent immutable computational project states.

The repository shall support:

```
get()  
get_current()  
create()
```

Once a model version becomes immutable, application code must not modify its persisted state.

A new model state creates a new model version.

19. CHANGE RECORD SERVICE

Every successful model mutation creates a ChangeRecord.

The record identifies:

```
object_id  
operation  
before_state  
after_state  
reason  
source  
revision_id
```

This creates the basis for:

- revision comparison
- undo analysis
- audit
- historical reconstruction
- redline verification

20. PROJECT SERVICE

`projects.py` becomes the first complete application service.

Its responsibilities include:

- project creation
- project retrieval
- project updates
- project authorization
- initial model creation
- initial revision creation
- initial event generation

21. PROJECT CREATION TRANSACTION

Project creation executes as:

```
CREATE ORGANIZATION CONTEXT
↓
VERIFY ACTOR
↓
CREATE PROJECT
↓
CREATE INITIAL MODEL VERSION
↓
CREATE INITIAL REVISION
↓
SET CURRENT MODEL
↓
SET CURRENT REVISION
↓
RECORD EVENT
↓
COMMIT
```

The API returns:

- `project_id`
- `revision_id`
- `model_version_id`

22. INITIAL PROJECT STATE

A newly created project shall have:

```
revision_number = 1  
model_version = 1  
status = ACTIVE
```

The initial project model may contain no building geometry.

The project itself is nevertheless a valid computational state.

23. DOMAIN ENTITY MODEL

The domain layer shall represent core objects independently of PostgreSQL.

Initial entities:

```
Project  
Requirement  
Site  
Building  
Level  
Space  
Element  
Assembly  
Constraint  
Relationship  
Revision  
ModelVersion  
AriaCommand  
Redline  
ValidationRun  
Drawing  
DocumentPackage  
AcceptanceRecord
```

Persistence models must not become the domain model by accident.

24. VALUE OBJECTS

Where appropriate, repeated concepts become value objects.

Initial candidates:

```
ProjectId  
RevisionId  
ModelVersionId  
ElementId  
Length
```

Area
Volume
Elevation
BoundingBox
Coordinate
Direction
UnitSystem
Jurisdiction

Value objects should enforce domain-valid representations.

25. UNIT SYSTEM

OneDraft supports at minimum:

METRIC
IMPERIAL

Internal computational units should be normalized.

Display units remain project configuration.

For example:

stored length → normalized unit
display length → project unit system

The geometry engine must not depend on arbitrary user display units.

26. DOMAIN ERRORS

The domain layer shall expose explicit errors.

Initial errors include:

ProjectNotFound
RevisionConflict
ModelVersionNotFound
ElementNotFound
InvalidCommand
ConstraintConflict
InvalidGeometry
InvalidLevel
InvalidRelationship
UnauthorizedProjectAccess
InvalidStateTransition

Infrastructure exceptions must not leak directly through the API.

27. API APPLICATION ENTRY POINT

`main.py` becomes the FastAPI application entry point.

It shall initialize:

configuration
logging
database
routes
exception handlers
health endpoints
middleware

The application shall expose:

`/api/v1`

28. HEALTH ENDPOINT

The application shall provide:

GET `/health`

Response:

```
{  
  "status": "ok"  
}
```

A deeper readiness endpoint may provide:

GET `/ready`

which verifies required infrastructure.

29. PROJECT API

The first executable endpoint shall be:

POST `/api/v1/projects`

The endpoint receives the Project Creation schema established by the OpenAPI contract.

Execution:

HTTP
↓
Schema validation

↓
Authentication
↓
ProjectService
↓
UnitOfWork
↓
Repositories
↓
PostgreSQL

30. PROJECT RETRIEVAL

The API shall provide:

GET /api/v1/projects/{project_id}

The response contains the current project state.

Historical versions are retrieved through revision-specific endpoints.

31. PROJECT LISTING

The API shall provide:

GET /api/v1/projects

Supported filters initially include:

status
created_after
created_before
updated_after
updated_before

Organization membership determines which projects are visible.

32. APPLICATION DEPENDENCIES

FastAPI dependencies shall provide:

current_user
organization_context
database_session
unit_of_work
authorization_context

Routes must not construct database connections manually.

33. AUTHORIZATION BOUNDARY

Authorization occurs before domain mutation.

The application verifies:

```
authenticated actor
+
organization membership
+
project membership
+
required scope
```

The domain service receives an already-authorized execution context.

34. IDEMPOTENCY

Mutating endpoints shall support:

Idempotency-Key

The application resolves the key before executing a duplicate operation.

Conceptually:

```
REQUEST
↓
HASH
↓
LOOKUP IDEMPOTENCY RECORD
↓
EXISTS → RETURN ORIGINAL RESULT
↓
NOT EXISTS → EXECUTE
```

35. OUTBOX PATTERN

Domain events shall use a transactional outbox.

The transaction writes:

```
domain state
+
revision
```

- +
change record
- +
event
- +
outbox message

in the same database transaction.

Only after commit may the asynchronous dispatcher publish the message.

36. EVENT MODEL

Initial events:

- ProjectCreated
- RequirementCreated
- ModelChanged
- RevisionCreated
- AriaCommandCreated
- AriaCommandCompleted
- RedlineCreated
- RedlineResolved
- ValidationRequested
- ValidationCompleted
- DrawingsRequested
- DrawingsCompleted
- DocumentPackageRequested
- DocumentPackageCompleted
- AcceptanceRecorded

Events are immutable.

37. EVENT PAYLOAD

An event shall contain:

- event_id
- event_type
- project_id
- revision_id
- model_version_id
- actor_id
- occurred_at
- payload

Where applicable:

- object_id
- command_id
- job_id

38. REDIS JOB LAYER

Redis becomes the initial asynchronous job broker.

The application publishes jobs for operations that should not block HTTP requests.

Initial job classes:

```
GEOMETRY
VALIDATION
DRAWING_GENERATION
DOCUMENT_GENERATION
ARIA_PROCESSING
```

39. WORKER

`worker.py` becomes the first executable asynchronous runtime.

The worker shall:

```
connect to Redis
consume jobs
load required project state
execute service
persist results
publish completion event
acknowledge job
```

A worker must be stateless between jobs.

40. JOB STATE

Every asynchronous job has:

```
job_id
project_id
job_type
status
progress
created_at
started_at
completed_at
error
result
```

Initial statuses:

QUEUED
RUNNING
VALIDATING
COMPLETED
FAILED
CANCELLED

41. JOB IDEMPOTENCY

Workers must tolerate duplicate delivery.

A job shall not produce duplicate model mutations merely because a message was delivered more than once.

Mutating work therefore requires:

job_id
command_id
expected_revision

and appropriate idempotency checks.

42. ARIA EXECUTION BOUNDARY

Aria does not directly manipulate PostgreSQL.

Aria produces:

intent
+
structured command

The application validates and executes that command.

The chain is:

```
USER
↓
ARIA
↓
COMMAND
↓
COMMAND VALIDATOR
↓
DOMAIN SERVICE
↓
UNIT OF WORK
↓
DATABASE
```

43. ARIA COMMAND EXAMPLE

A natural-language request:

Move the rear bedroom wall 300 mm north while keeping the exterior envelope unchanged.

is transformed into:

operation:
MOVE_ELEMENT

target:
element UUID

parameters:
distance = 300
unit = mm
direction = NORTH

preserve:
exterior envelope

The application remains the authority that determines whether the operation is valid.

44. COMMAND EXECUTION

A command execution service shall:

```
load project
load current revision
verify expected revision
load targets
load constraints
validate operation
calculate mutation
apply mutation
record changes
create model version
create revision
create event
commit
```

The returned result identifies the new computational state.

45. MODEL MUTATION RULE

A model mutation must never directly overwrite historical state.

Instead:

```
Revision 14
  ↓
command
  ↓
Revision 15
```

The previous state remains reconstructable.

46. GEOMETRY DISPATCH

Geometry computation is asynchronous when computationally expensive.

The application records the intended model state and dispatches:

GeometryJob

containing:

```
project_id
model_version_id
element_id
geometry_version
```

47. STALE GEOMETRY PROTECTION

The worker verifies that the target model version remains relevant.

If the project has advanced beyond the expected state:

STALE_JOB

is recorded.

The worker must not silently replace current geometry with stale output.

48. OBJECT STORAGE

Large computational artifacts shall not be placed directly into PostgreSQL.

Object storage shall contain:

- geometry payloads
- generated PDFs
- drawing exports
- rendered previews
- source documents
- code artifacts
- validation artifacts

PostgreSQL stores references and hashes.

49. OBJECT STORAGE REFERENCE

A persisted artifact should identify:

- storage_provider
- bucket
- object_key
- content_type
- content_hash
- size
- created_at

The database becomes the provenance index.

Object storage becomes the artifact store.

50. COMPLIANCE SERVICE

The compliance service consumes:

- project model
- jurisdiction
- code manifest

and produces:

- applicable rules
- compliance checks
- validation inputs

The service does not directly modify architectural geometry.

51. VALIDATION SERVICE

The validation service receives:

```
project_id
revision_id
model_version_id
code_manifest_id
```

It evaluates:

```
CODE
SPATIAL
ACCESSIBILITY
DOCUMENT
COORDINATION
```

and produces ValidationResults.

52. VALIDATION JOB

The API creates:

```
ValidationRun
```

with:

```
status = QUEUED
```

The worker then executes:

```
QUEUED
↓
RUNNING
↓
VALIDATING
↓
COMPLETED
```

The results remain attached to the exact revision and model version tested.

53. DRAWING SERVICE

The drawing service converts the computational model into documentation views.

The initial pipeline is:

```
MODEL VERSION
↓
```


VIEW GENERATION
↓
DRAWING GENERATION
↓
SHEET COMPOSITION
↓
ARCH D OUTPUT

The drawing is derived from the model.

It is never the authoritative model.

54. DOCUMENT SERVICE

The document service assembles:

drawings
schedules
notes
validation information
revision information

into a final document package.

The package identifies:

project_id
revision_id
model_version_id
code_manifest_id
validation_run_id

55. REDLINE SERVICE

Redlines enter the system as explicit change requests.

The lifecycle is:

OPEN
↓
ANALYZING
↓
PROPOSED
↓
IMPLEMENTED
↓
VALIDATED
↓
RESOLVED

A redline cannot become resolved merely because text was generated.

A corresponding model operation must exist.

56. ACCEPTANCE SERVICE

Acceptance occurs only against a specific:

revision
model version
document package
code manifest

The acceptance record therefore becomes an explicit historical statement about a particular generated package.

57. APPLICATION EVENT FLOW

A normal design modification becomes:

```
CLIENT
↓
POST /aria/requests
↓
ARIA ORCHESTRATION
↓
ARIA COMMAND
↓
COMMAND VALIDATION
↓
PROJECT MODEL SERVICE
↓
UNIT OF WORK
↓
POSTGRESQL
↓
REVISION CREATED
↓
OUTBOX EVENT
↓
REDIS
↓
WORKER
↓
GEOMETRY / VALIDATION / DRAWING
↓
DOCUMENTATION
```

58. TRANSACTIONAL INTEGRITY

No service may perform:

database mutation
+
external side effect

without accounting for transaction boundaries.

Database state must first become durable.

Asynchronous external work proceeds from the committed event/job state.

59. FAILURE MODEL

Failures must be explicit.

Examples:

VALIDATION_FAILED
GEOMETRY_FAILED
DRAWING_FAILED
DOCUMENT_GENERATION_FAILED
COMMAND_REJECTED
REVISION_CONFLICT
CONSTRAINT_CONFLICT
STALE_JOB

A failed asynchronous job must not be represented as successful project state.

60. RETRY POLICY

Retryable infrastructure failures may be retried.

Domain failures must not be blindly retried.

For example:

database connection failure
→ retryable

temporary object-storage failure
→ retryable

invalid geometry
→ domain failure

constraint conflict
→ domain failure

revision conflict
→ domain failure

61. LOGGING

Every request and job receives a correlation identifier.

Logs shall support:

request_id
project_id
command_id
revision_id
model_version_id
job_id
event_id

This permits reconstruction of execution flow across services.

62. OBSERVABILITY

The initial implementation shall provide:

structured logs
health checks
job status
request identifiers
error tracking
basic execution timing

Future observability can add:

metrics
distributed tracing
performance dashboards

without changing the domain model.

63. CONFIGURATION

Runtime configuration shall be environment-driven.

Initial variables include:

DATABASE_URL
REDIS_URL

OBJECT_STORAGE_ENDPOINT
OBJECT_STORAGE_BUCKET
OBJECT_STORAGE_ACCESS_KEY
OBJECT_STORAGE_SECRET_KEY
API_ENVIRONMENT
LOG_LEVEL
ARIA_PROVIDER

Secrets must never be committed to source control.

64. DEVELOPMENT ENVIRONMENT

The local development stack shall support:

PostgreSQL
Redis
API
Worker
Object Storage

Conceptually:

docker compose
↓
PostgreSQL
Redis
Object Storage
API
Worker

65. API STARTUP

Startup sequence:

LOAD CONFIG
↓
INITIALIZE LOGGING
↓
INITIALIZE DATABASE
↓
VERIFY DATABASE
↓
INITIALIZE REDIS
↓
INITIALIZE SERVICES
↓
REGISTER ROUTES
↓
START API

The API should fail fast if required infrastructure is unavailable.

66. WORKER STARTUP

Worker startup:

```
LOAD CONFIG
↓
INITIALIZE DATABASE
↓
INITIALIZE REDIS
↓
INITIALIZE OBJECT STORAGE
↓
REGISTER JOB HANDLERS
↓
BEGIN CONSUMPTION
```

67. TESTING PYRAMID

Testing begins at the domain boundary.

The test hierarchy is:

```
UNIT TESTS
↓
SERVICE TESTS
↓
REPOSITORY TESTS
↓
API TESTS
↓
WORKER TESTS
↓
INTEGRATION TESTS
↓
END-TO-END TEST
```

The full system test established in v0.1 remains the primary integration objective.

68. DOMAIN UNIT TESTS

The first domain tests shall verify:

```
revision incrementing
constraint enforcement
command validation
```

- element relationships
- model mutation
- change records
- state transitions

These tests must run without PostgreSQL.

69. REPOSITORY TESTS

Repository tests verify:

- CRUD operations
- foreign-key relationships
- revision retrieval
- project tenancy boundaries
- transaction rollback
- historical state preservation

These tests use a real PostgreSQL test environment rather than relying exclusively on mocks.

70. API TESTS

API tests verify:

- authentication
- authorization
- request validation
- response schemas
- error handling
- idempotency
- revision conflicts
- project creation
- project retrieval

71. WORKER TESTS

Worker tests verify:

- job consumption
- job state transitions
- duplicate delivery handling
- failure handling
- retry behavior
- stale model detection
- result persistence

72. FIRST EXECUTABLE TEST

The first executable test shall be:

POST /api/v1/projects

followed by:

GET /api/v1/projects/{project_id}

The test must prove:

HTTP
↓
application service
↓
Unit of Work
↓
repository
↓
PostgreSQL
↓
response

73. SECOND EXECUTABLE TEST

The second test shall execute:

CREATE PROJECT
↓
CREATE BUILDING
↓
CREATE LEVEL
↓
CREATE SPACE
↓
CREATE WALL

The test must verify that the resulting objects are linked correctly.

74. THIRD EXECUTABLE TEST

The third test shall execute:

CREATE MODEL
↓
MOVE ELEMENT
↓
CREATE REVISION

↓
VERIFY PRIOR REVISION
↓
VERIFY CURRENT REVISION

The test must demonstrate historical preservation.

75. FOURTH EXECUTABLE TEST

The fourth test shall execute:

ARIA REQUEST
↓
COMMAND
↓
VALIDATION
↓
MODEL MUTATION
↓
REVISION
↓
EVENT
↓
JOB

This becomes the first proof that Aria operates through the application boundary.

76. FIFTH EXECUTABLE TEST

The fifth test shall execute:

MODEL
↓
DRAWING GENERATION
↓
VALIDATION
↓
DOCUMENT PACKAGE

The generated package must reference the exact model and revision from which it was generated.

77. SECURITY BOUNDARY

The following remain prohibited:

Aria → SQL
Aria → PostgreSQL

Aria → filesystem replacement
Aria → production configuration
Aria → permission modification
Aria → billing mutation

All privileged operations require application capabilities.

78. DOMAIN / INFRASTRUCTURE SEPARATION

The architecture shall maintain:

DOMAIN

separate from:

DATABASE
HTTP
REDIS
OBJECT STORAGE
AI PROVIDERS

This allows infrastructure to change without redefining the OneDraft computational model.

79. PROVIDER ADAPTERS

External infrastructure shall be accessed through adapters.

Initial interfaces:

AIPProvider
GeometryProvider
ObjectStorageProvider
JobQueue
CodeSourceProvider
DocumentRenderer

The domain layer depends upon interfaces.

Concrete providers belong to infrastructure.

80. ARIA PROVIDER ADAPTER

Aria orchestration shall expose an internal interface similar to:

```
interpret()  
propose()  
explain()  
validate_intent()
```

The provider adapter may change without changing the public OneDraft API.

81. GEOMETRY PROVIDER

The geometry subsystem shall eventually support:

```
create_geometry  
modify_geometry  
validate_geometry  
compute_intersections  
compute_bounds  
generate_section_geometry  
generate_projection
```

The initial implementation may use a simplified computational geometry engine, but the service boundary must remain stable.

82. DRAWING PROVIDER

The documentation engine shall eventually support:

```
generate_plan  
generate_elevation  
generate_section  
generate_detail  
generate_schedule  
compose_sheet  
export_pdf
```

Each operation identifies the model version used.

83. OBJECT STORAGE PROVIDER

The storage interface shall support:

```
put()  
get()  
exists()  
delete()  
hash()
```

Physical storage implementation remains infrastructure-specific.

84. CODE SOURCE PROVIDER

The compliance layer shall retrieve regulatory source material through a controlled provider.

The system records:

source identity
version
effective date
retrieval timestamp
content hash

The resulting Code Manifest remains the authoritative project-specific regulatory snapshot.

85. DOMAIN SERVICE REGISTRY

The application runtime shall initialize:

ProjectService
ModelService
CommandService
RevisionService
RedlineService
ComplianceService
ValidationService
DrawingService
DocumentService
AcceptanceService
AriaService

Services receive their dependencies through explicit construction.

86. DEPENDENCY INJECTION

Dependencies shall flow inward:

API
↓
APPLICATION SERVICES
↓
DOMAIN

Infrastructure is injected into the application layer.

The domain must not instantiate:

PostgreSQL
Redis
HTTP clients
AI providers
object-storage clients

itself.

87. SERVICE TRANSACTION POLICY

Application services determine transaction boundaries.

A service operation that changes authoritative project state must execute through the Unit of Work.

Read-only operations may use controlled read sessions.

Long-running computation must not hold a PostgreSQL transaction open unnecessarily.

88. LONG-RUNNING OPERATIONS

The following shall be asynchronous:

large geometry generation
full compliance validation
large drawing sets
document package generation
complex Aria processing

The HTTP API returns:

job_id
status

rather than holding the request open.

89. COMMAND / JOB DISTINCTION

A command represents:

WHAT SHOULD HAPPEN

A job represents:

WORK REQUIRED TO COMPLETE IT

These concepts remain separate.

Example:

```
GenerateDrawingsCommand
  ↓
DrawingGenerationJob
  ↓
DrawingService
```

90. EVENT / JOB DISTINCTION

An event represents:

WHAT HAS HAPPENED

A job represents:

WHAT MUST BE DONE

Therefore:

ModelChanged

is an event.

GenerateAffectedDrawings

is a job.

91. RECONSTRUCTION REQUIREMENT

The service layer must preserve the v0.1 reconstruction guarantee.

Given:

```
Project_ID
Revision_ID
Model_Version_ID
Code_Manifest_ID
```

the application must be able to retrieve the authoritative state required to reconstruct the project.

92. CURRENT STATE PROJECTION

The current project state is a projection of the latest committed model version.

The database may maintain:

current_revision_id
current_model_version_id

for fast access.

Historical versions remain immutable.

93. REVISION FINALIZATION

A revision progresses through controlled states.

Initial state model:

DRAFT
↓
VALIDATING
↓
VALIDATED
↓
DOCUMENTED
↓
ACCEPTED

Failure states may include:

REJECTED
INVALID
FAILED

State transitions must be explicit.

94. ACCEPTANCE FINALITY

Once a customer accepts a document package, the acceptance record identifies the exact package.

The system must not silently replace the accepted package with a later revision.

A subsequent design modification creates a new revision.

95. AUDIT INTEGRATION

Every significant application operation shall produce an auditable event.

At minimum:

project creation
model mutation

revision creation
Aria command
redline creation
redline resolution
validation
drawing generation
document generation
acceptance

96. APPLICATION OBSERVABILITY CHAIN

The complete execution trace should be correlatable:

```
REQUEST_ID
↓
COMMAND_ID
↓
REVISION_ID
↓
MODEL_VERSION_ID
↓
EVENT_ID
↓
JOB_ID
↓
ARTIFACT_HASH
```

This provides the technical provenance chain for generated project documentation.

97. INITIAL FILE IMPLEMENTATION ORDER

The executable files shall be implemented in this order:

1. database.py
2. repositories.py
3. unit_of_work.py
4. commands.py
5. projects.py
6. main.py
7. worker.py

This order follows dependency direction.

98. DATABASE.PY

`database.py` establishes:

- PostgreSQL engine
- session factory
- session dependency
- transaction lifecycle
- health check

It is the lowest-level application persistence entry point.

99. REPOSITORIES.PY

`repositories.py` establishes:

- typed repository interfaces
- PostgreSQL repository implementations
- query boundaries
- persistence mapping

It prevents route handlers from becoming database code.

100. UNIT_OF_WORK.PY

`unit_of_work.py` establishes:

- transaction boundary
- repository collection
- commit
- rollback
- event persistence
- outbox persistence

It becomes the authoritative mechanism for transactional domain mutations.

101. COMMANDS.PY

`commands.py` establishes:

- command schemas
- command validation

command identity
expected revision
command execution metadata

It becomes the foundation for both HTTP-driven and Aria-driven model operations.

102. PROJECTS.PY

`projects.py` establishes the first complete application service.

It proves:

authentication
↓
authorization
↓
command
↓
Unit of Work
↓
repository
↓
PostgreSQL
↓
event
↓
response

103. MAIN.PY

`main.py` assembles the executable API.

It shall:

create FastAPI application
register dependencies
register project routes
register health routes
register exception handlers
initialize middleware

Additional route groups are added incrementally.

104. WORKER.PY

`worker.py` establishes asynchronous execution.

It shall:

```
initialize infrastructure
consume queue
dispatch job
execute service
record result
publish completion
```

The worker must never directly manipulate domain state outside the established service boundary.

105. FIRST RUNNING SYSTEM

After these artifacts are implemented, OneDraft must be capable of running as:

```
docker compose up
```

with:

```
PostgreSQL
Redis
API
Worker
Object Storage
```

available as development services.

106. FIRST LIVE REQUEST

The first live request shall be:

```
POST /api/v1/projects
```

The resulting project must appear in PostgreSQL.

The returned identifiers must be valid:

```
project_id
revision_id
model_version_id
```

107. FIRST LIVE MUTATION

The first live model mutation shall prove:

```
CREATE ELEMENT
```

↓
MODIFY ELEMENT
↓
CREATE REVISION
↓
PERSIST CHANGE RECORD

The previous revision must remain queryable.

108. FIRST LIVE ASYNC JOB

The first live asynchronous operation shall prove:

REQUEST
↓
JOB CREATED
↓
REDIS
↓
WORKER
↓
PROCESSING
↓
RESULT
↓
JOB COMPLETED

109. FIRST LIVE ARIA OPERATION

The first live Aria operation shall demonstrate:

NATURAL LANGUAGE
↓
STRUCTURED INTENT
↓
COMMAND
↓
APPLICATION VALIDATION
↓
DOMAIN MUTATION
↓
REVISION

The AI layer therefore becomes an intelligent client of the OneDraft domain rather than the domain itself.

110. FIRST COMPLETE EXECUTION PATH

The first complete executable path shall become:

USER
↓
API
↓
PROJECT
↓
MODEL
↓
ARIA
↓
COMMAND
↓
REVISION
↓
EVENT
↓
JOB
↓
WORKER
↓
VALIDATION
↓
DRAWING
↓
DOCUMENT
↓
ACCEPTANCE

This is the first vertical slice of the OneDraft platform.

111. VERTICAL SLICE PRINCIPLE

Development shall proceed by executable vertical slices rather than completing isolated infrastructure indefinitely.

Each slice must cross:

API
DOMAIN
DATABASE
EVENT
WORKER

where applicable.

The system should become progressively executable at every stage.

112. MVP VERTICAL SLICE

The first complete slice is:

```
CREATE PROJECT
↓
CREATE BUILDING
↓
CREATE LEVEL
↓
CREATE SPACE
↓
CREATE WALL
↓
MOVE WALL
↓
CREATE REVISION
↓
GENERATE PLAN
↓
VALIDATE
↓
GENERATE DOCUMENT
```

This slice establishes the minimum computational drafting loop.

113. REDLINE VERTICAL SLICE

The second major slice is:

```
DRAWING
↓
REDLINE
↓
ARIA INTERPRETATION
↓
COMMAND
↓
MODEL CHANGE
↓
REVISION
↓
VALIDATION
↓
DRAWING UPDATE
↓
REDLINE RESOLUTION
```

This establishes the iterative drafting workflow.

114. REGULATORY VERTICAL SLICE

The third major slice is:

PROJECT JURISDICTION
↓
CODE SOURCES
↓
CODE MANIFEST
↓
APPLICABLE RULES
↓
VALIDATION
↓
RESULTS
↓
DOCUMENTATION

This establishes jurisdiction-aware design verification.

115. DOCUMENT PROVENANCE SLICE

The fourth major slice is:

MODEL VERSION
↓
REVISION
↓
CODE MANIFEST
↓
VALIDATION RUN
↓
DRAWINGS
↓
DOCUMENT PACKAGE
↓
HASH
↓
ACCEPTANCE

This establishes reproducible document provenance.

116. ENGINEERING DEFINITION OF DONE

This implementation stage is complete when:

API starts

Database connects

Redis connects
Worker starts
Project can be created
Project can be retrieved
Model can be mutated
Revision is created
Historical state is preserved
Events are recorded
Jobs are dispatched
Workers execute jobs
Aria commands pass through the application boundary
Validation can execute
Drawings can be generated
Documents can be packaged
Acceptance can be recorded

117. NEXT MACHINE ARTIFACT PACKAGE

The next implementation package shall now be generated as:

apps/api/main.py
apps/api/routes/projects.py
apps/worker/worker.py
packages/domain/commands.py
packages/persistence/database.py
packages/persistence/repositories.py
packages/persistence/unit_of_work.py

These are the immediate executable artifacts following the established v0.1 contract.

118. IMPLEMENTATION SEQUENCE AFTER THIS SPECIFICATION

Once these files exist, implementation proceeds:

APPLICATION RUNTIME
↓
COMMAND EXECUTION
↓
MODEL SERVICES
↓
EVENT / OUTBOX
↓
REDIS JOBS
↓
GEOMETRY ENGINE
↓
COMPLIANCE ENGINE
↓
VALIDATION ENGINE
↓
DRAWING ENGINE
↓
DOCUMENT ENGINE
↓
ARIA ORCHESTRATION
↓
REDLINE LOOP
↓
ACCEPTANCE
↓
END-TO-END SYSTEM

119. ARCHITECTURAL PRINCIPLE

OneDraft remains fundamentally:

COMPUTATIONAL PROJECT MODEL
+
ARIA INTELLIGENCE
+
GEOMETRY ENGINE
+
REGULATORY ENGINE
+
DOCUMENT ENGINE
+
VERSION CONTROL
+
CUSTOMER REVIEW

The executable application layer is the connective tissue that makes those components operate as one system.

120. SPECIFICATION STATUS

ONEDRAFT EXECUTABLE APPLICATION & DOMAIN SERVICE LAYER v0.3

STATUS: ESTABLISHED

The database schema and OpenAPI contract remain the authoritative persistence and machine-interface definitions.

The executable service layer now defines how those contracts become a running application.

The immediate implementation artifacts are:

- `commands.py`
- `database.py`
- `repositories.py`
- `unit_of_work.py`
- `main.py`
- `projects.py`
- `worker.py`